How Invariant is Code Generation to Graph Serialization? Measuring and Localizing the Fragility

Ram Kedem 206844144 **Yehonatan Barel** 208646422 **Ido Azoulay** 212710958 **Dolev Abudi** 323096099

Abstract

Language models often answer graph questions better by writing code than by answering directly. If the answer depends on how the graph is serialized, however, a correct answer for one serialization does not guarantee a correct answer for another. **We ask:** ***does solving a graph problem using code generation return the same answer under equivalent serializations of the same graph, and if not, at which step does the fragility enter?*** Across four models and two datasets, changing one aspect of the serialization at a time, we find that code generation is far more robust to the serialization, changing its answer two to five times less often than direct answering, a gap that grows with graph size, yet no condition in which the model reads the graph is fully invariant. The fragility enters mostly in the program the model writes, not in its copy of the graph. Keeping the graph out of the prompt nearly removes the sensitivity at no cost in accuracy.

1 Introduction

Large language models typically receive graphs as serialized text. The choice of serialization affects both accuracy (Fatemi et al., 2024) and permutation invariance: renaming nodes or reordering edges leaves the graph and the question unchanged, yet can change the answer (Ge et al., 2025; Herbst et al., 2026). Code generation has proved one of the strongest ways to answer graph questions (Cai et al., 2024; Finkelshtein et al., 2026), but higher accuracy does not necessarily mean invariance across serializations (Herbst et al., 2026).

We send each graph question to the model in three ways. In direct answering (M1), the model reads the serialized graph and replies with the answer. In code from text (M2), it reads the same graph and replies with a Python program that copies the graph and computes the answer when we run it. In Graph-as-Code (M3), it writes the program from the question alone, and we pass the graph to the program as data when we run it. Comparing M1 with M2 shows how much code generation reduces the sensitivity to serialization. The M2 programs show where the remaining sensitivity enters, in the model's copy of the graph or in the program it writes. M3 tests whether keeping the graph out of the prompt removes the sensitivity without losing accuracy.

Our contributions are as follows. (i) A per-instance, single-factor invariance measurement of *executed* code over four models and two datasets. (ii) Evidence that when the answer changes, the cause is rarely a wrong copy of the graph but the program the model writes. (iii) We introduce a mitigation that keeps the graph out of the prompt and nearly removes the sensitivity at no cost in accuracy, provided the prompt states the graph type.

2 Related Work

Graph reasoning from text. NLGraph (Wang et al., 2023) and GraphQA (Fatemi et al., 2024) showed that LLMs solve small graph problems posed in text, but unreliably: on GraphQA the best and worst encodings of the same graphs differ by 4.8 to 61.8 points for PaLM 62B. Erdős (Guo et al., 2025) extends evaluation to 50 tasks.

Serialization invariance. The order of a graph's description alone shifts accuracy (Ge et al., 2025), and Herbst et al. (2026), varying node labels, computational structure and syntax one at a time, find that each factor changes outputs. Their models answer directly, reading NetworkX code as text rather than running it, and their per-example measure covers numeric answers only. We measure per-instance agreement on executed code, for every answer type.

Computing with code. Program-aided reasoning hands the computation to an interpreter (Gao et al., 2023). CodeGraph (Cai et al., 2024) has GPT-3.5 write Python for GraphQA tasks, and across six encodings its best-worst accuracy gap

is smaller than under chain-of-thought on every task. Finkelshtein et al. (2026) find code generation the strongest interaction mode for node classification on text-rich graphs. Neither shows invariance: Finkelshtein et al. (2026) use a single serialization, and CodeGraph compares aggregate accuracy over encodings that change several factors at once. Its one-shot example also supplies the solving function, so its model mostly copies the graph instead of writing the solving step, where we find the failures.

3 Methodology

Each *instance* is a graph, a question about it and the correct answer. We write each graph in several equivalent serializations, pose each one in three interaction modes, and check if the answers stay the same and whether they are correct.

3.1 Data

We use two benchmarks, one with basic graph questions and one with harder algorithmic tasks.

GraphQA: This benchmark (Fatemi et al., 2024) asks seven types of basic question, such as edge existence, node degree and cycle detection, about small random graphs (5-20 nodes) built by seven graph generators. We follow its recipe to generate 100 graphs and ask one question of each type on every graph, giving 700 instances.

Erdős: This published suite (Guo et al., 2025) covers 50 harder algorithmic tasks and is the one Herbst et al. (2026) used. We take 12 of its tasks, such as shortest path, triangle counting and bridges, with 100 test graphs each (4-34 nodes).

3.2 Interaction Modes

Direct answering (M1): The model reads the serialized graph and the question and answers directly. This is the known fragile baseline (Herbst et al., 2026).

Code from text (M2): The model gets the same input and writes a Python program that we execute. The program must start by writing out the graph as a list of nodes and a list of edges, so we see exactly what the model copied.

Graph-as-Code (M3): In this mode, adapted from Finkelshtein et al. (2026), the model sees only the task and the question and writes code over a list of nodes and a list of edges that it never sees. We fill these lists with the serialized graph at execution.

Both code modes run in two library arms: native, limited to the Python standard library so that the model writes the algorithm itself, and NetworkX allowed. Figure A1 in Appendix A shows the full prompts of the three modes for one instance.

3.3 Serialization Factors

The reference serialization is the dataset's own edge list. Every other serialization changes exactly one of four factors, so that a change in the answer can be attributed to that factor. **Relabeling** applies a random permutation of the node labels to the graph and to the question and maps node-valued answers back. **Order** lists the edges in three orderings: sorted by the first endpoint, by the second, or shuffled. **Structure** adds two serializations: an adjacency list, and, for undirected graphs, an edge list in which each edge is written twice, as (u, v) and (v, u). **Syntax** writes the same edge list as plain text, JSON or NetworkX code (Erdős: JSON and NetworkX code only).

4 Experimental Setup

4.1 Models

We test four models of different sizes: Qwen3-8B (Alibaba, 8B parameters), Gemma 4 31B (Google, 31B), DeepSeek-V4-Flash (284B, of which 13B are active for each token) and DeepSeek-V3.1 (671B, 37B active).

Inference: All four models run through the same API gateway with the same settings: temperature 0, at most 4,096 output tokens, and reasoning turned off, so that direct answering stays a direct answer rather than a hidden chain of thought (Appendix C). Generated programs run in a separate process with a time limit.

Scale: Each model answers every serialization of every instance in all five conditions (Section 3.2): 44,625 answers per model on GraphQA and 71,230 on Erdős.

4.2 Metrics

Terms: A *factor* is the kind of change: relabeling, edge order, structure or syntax (Section 3.3). A *serialization* is one concrete text of a graph. A *tuple* collects the answers one model gives, in one mode, to the serializations of one instance under one factor (Figure C1). Under relabeling, for example, a tuple holds four answers: to the reference labeling and to three random relabelings. Table C1 in Appendix C gives the tuple sizes.

Correctness: An answer is correct when it equals the true answer. When a question has several correct answers, for example two equally short paths, any of them is correct. Real-valued answers

are compared with a small tolerance (Appendix C). A crash, a timeout or an unreadable reply counts as wrong.

Performance: Accuracy is the share of all answers that are correct. Robust accuracy is the share of tuples in which every answer is correct.

Invariance: A tuple *flips* when at least one of its answers differs from the others. When a question has several correct answers, they count as the same answer, and a missing answer counts as an answer of its own. The flip rate is the share of tuples that flip. Crossing flips with correctness divides the tuples into six buckets (Table 1): (1) the same correct answer on every serialization, (2) the same wrong answer, (3) different answers that are all correct, (4) a flip in which every serialization is answered, (5) a flip with a crash, and (6) no answer on any serialization. Buckets 4 and 5 are the flips, and bucket 2 holds errors that invariance cannot see.

Weighted flips: The flip rate treats every flip alike, whether one answer deviates or all of them differ. We therefore define two measures on a tuple with answers $a_1, \ldots, a_n$ and true answer y. D is the fraction of answer pairs that differ, and S is the range of the answers relative to the true answer:

$$D = \frac{1}{\binom{n}{2}} \sum_{1 \le i < j \le n} \mathbb{1}\left[a_i \neq a_j\right],$$

$$S = \frac{\max_i a_i - \min_i a_i}{|y|}$$

D is 0 when all answers agree and 1 when all differ: (1, 2, 2, 2) has $D = 0.5$ and (1, 2, 3, 4) has $D = 1$. S applies to numeric answers, and $S = 1$ means the answers span the whole true value, as when a count doubles. The mean of D over flipped tuples is the *severity* of the flips.

Localization: We trace M2 failures to one of three steps: the copy of the graph (the program declares a wrong graph), a crash (it gives no answer) or the program's solving step (otherwise). Each wrong answer is assigned the first of these steps that failed, and each flipped tuple the step at which its serializations diverge.

Statistics: Every rate is computed per dataset, mode and factor, with the four models weighted equally. Brackets are 95% confidence intervals with errors clustered by graph, and whiskers are 95% bootstrap intervals over graphs (Appendix C).

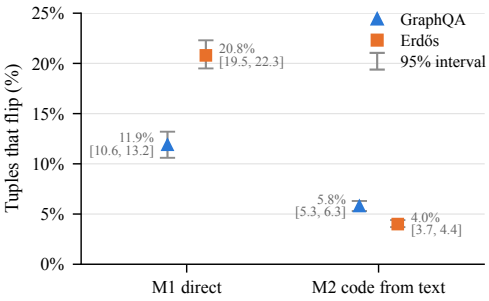


Figure 1: Flip rate under relabeling and edge order, with M2 averaged over plain Python and NetworkX. Whiskers are 95% bootstrap intervals over graphs.

5 Results and Discussion

5.1 Does executed code keep its answer, and more often than direct answering?

Executed code flips far less often than direct answering (Figure 1), but is still not invariant. Under relabeling and edge order, direct answers (M1) flip on 11.9% of GraphQA and 20.8% of Erdős tuples. Code from text (M2) flips on 4.8% and 3.5% in plain Python and 6.8% and 4.5% with NetworkX. Code flips significantly less (McNemar's test) in 14 of 16 model × dataset × code library cells. Both exceptions are Gemma 4 31B on GraphQA, whose direct answers are already the most stable (3.0%). In a logistic regression with model, task and factor held fixed, M2's plain-Python code has 0.47 [0.41, 0.54] times direct answering's odds of a flip on GraphQA and 0.20 [0.17, 0.22] on Erdős, and NetworkX code 0.52 [0.44, 0.61] and 0.31 [0.27, 0.36]. Figures B2 and B3 (Appendix B) give each model.

On Erdős, code's flips are also milder (Figure B1). Four in five plain-Python flips have a single serialization out of line ($D = 0.5$), against two in five for direct answering. Severity is 0.54 against 0.71, so weighting by D widens the gap from 6× to 8×. On GraphQA severity barely differs (0.56 against 0.62).

The gap grows with the graph (Figure 2). From graphs with up to 10 edges to graphs with more than 40, direct answering's flip rate rises from 7.7% to 23.1% on GraphQA and from 6.1% to 36.2% on Erdős, while code stays between 1.5% and 8.2%. In a logistic regression with task and model held fixed, each doubling of the edge count multiplies direct answering's odds of a flip by 1.60 [1.46, 1.74] on GraphQA and 2.44 [2.19, 2.71] on Erdős, and plain-Python code's by 0.96 [0.86, 1.08] and 1.52 [1.34, 1.72].

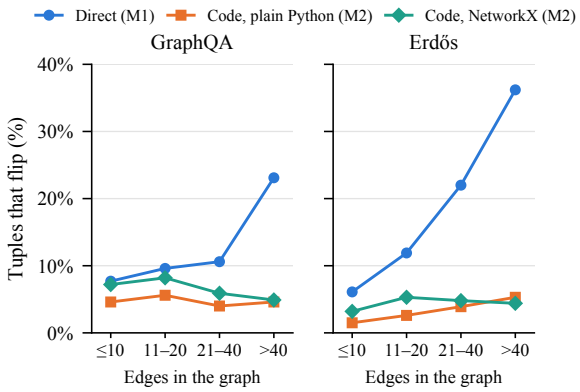


Figure 2: Share of tuples that flip under relabeling and edge order, by the number of edges, four models pooled.

	GraphQA			Erdős		
Bucket	Dir.	Py	NX	Dir.	Py	NX
(1) Same, correct	76.7	87.2	85.4	74.9	91.3	85.8
(2) Same, wrong	11.4	8.1	7.7	0.5	0.6	0.3
(3) Different, all correct	0.0	0.0	0.0	3.8	4.5	4.3
(4) Flip, all answered	11.5	4.7	5.5	17.6	2.7	0.9
(5) Flip with a crash	0.3	0.0	1.2	3.2	0.9	3.6
(6) No answer on all	0.0	0.0	0.1	0.0	0.0	5.0

Table 1: Tuples by bucket under relabeling and edge order, % of tuples, four models pooled. Buckets 4 and 5 are the flips. Dir. = M1. Py and NX = M2 in plain Python and with NetworkX.

Although we use temperature 0, the same prompt asked again can get a different answer, so part of every flip rate is noise. Only Qwen3-8B's code clearly exceeds this repeat floor (Appendix C).

A stable answer is not always a correct one (Table 1, per model in Table B1). On GraphQA, 8.1% of plain-Python tuples give the same wrong answer on every serialization. Most of them come from one question, which nodes are not connected to a given node, where DeepSeek-V3.1 and DeepSeek-V4-Flash both list the nodes it cannot reach instead of those without a direct edge to it. Invariance cannot see these errors, so we report robust accuracy beside it (Table 3).

5.2 Which factor flips the answer?

Edge structure flips plain-Python answers the most, and NetworkX-code syntax flips NetworkX answers by crashing the programs. Relabeling and order permute the same text, and code barely reacts to them: 3.5-7.2% of code tuples flip (Table 2, per model in Table B2). In the same logistic regression, structure raises the odds of a flip relative to relabeling 2.18-fold [1.99, 2.39] on GraphQA and 1.68-fold [1.55, 1.83] on Erdős, and syntax 1.42-fold [1.29, 1.56] and 1.45-fold [1.35, 1.56].

	GraphQA			Erdős		
Factor	Dir.	Py	NX	Dir.	Py	NX
Relabeling	**12.6**	5.6	7.2†	**22.1**	3.6	4.7
Order	**11.1**	4.0	6.3†	**19.6**	3.5	4.3
Structure	**19.2**	**18.5**†	8.0	**19.5**	**13.7**	**10.2**
Syntax	**14.9**	4.8	**13.6**†	**18.9**	3.8	**16.6**†

Table 2: Flip rate by factor (% of tuples, models pooled). Bold: 10% or more. †: some model's code flips at least 1 point more than its direct answers. Dir. = M1. Py, NX = M2 in plain Python and NetworkX.

Structure: Listing each undirected edge in both directions makes plain-Python programs count it twice, and 18.5% of GraphQA and 13.7% of Erdős tuples flip. On GraphQA node degree, 176 of Qwen3-8B's 200 answers are exactly double the true degree. These flips are far off, not just different: their median spread S is 1.0 on both datasets, and 93% (GraphQA) and 49% (Erdős) contain an exact doubling. A NetworkX `Graph` merges the duplicates, so NetworkX code flips less (8.0% and 10.2%). Accuracy follows: structure costs plain-Python code 9.8 points on GraphQA and 6.7 on Erdős, while relabeling and order cost code at most 0.7.

Syntax: When the graph arrives as NetworkX code, NetworkX programs crash on some serializations and not on others, and 13.6% and 16.6% of tuples flip. Qwen3-8B's programs leave out the import, and DeepSeek-V4-Flash's use a `G` they never define, both as if the prompt's code had already run. Without these two crashes, which cause 51% (GraphQA) and 71% (Erdős) of the flips, NetworkX code would flip on 6.6% and 4.8%, about as often as under relabeling and order. The syntax costs NetworkX code 6.2 points of accuracy on Erdős, while plain-Python code is unaffected (4.8% and 3.8% flip).

5.3 Which failures accompany a flip?

Most flips start in the program the model writes, and few in its copy of the graph. The graph a program declares matches the input in 98.1-99.9% of code answers. Among failed plain-Python answers, the solving step accounts for 97.8-99% on GraphQA and 77-81% on Erdős, while with NetworkX crashes account for 63-97% of Erdős failures. A wrong copy is at most about 13% of failures, and it does not reveal itself: 64% (GraphQA)

and 60% (Erdős) of programs that declare a wrong graph still answer correctly.

Failures are not flips, though, so Figure B4 attributes each flip itself to where its serializations diverge. On GraphQA, 95.5% of plain-Python flips start in the program. On Erdős the program still leads (61.5%), but a crash starts 20.9%, mostly DeepSeek-V4-Flash's, and a wrong copy 17.6%: 35 of DeepSeek-V3.1's 40 flips come from a wrong copy of a long edge list.

The M3 control points the same way: under edge order, Graph-as-Code runs one fixed program on every ordering and flips on 0.0-0.1% of tuples, against 3.5-6.3% for M2 (significantly more in 12 of 16 cells). Since M3 also hides the graph from the model, this is a diagnostic, not a causal estimate.

When code flips, the programs for different serializations often answer different readings of the question (Figure B5). Under relabeling and order, on the two GraphQA questions that ask which nodes are, or are not, connected to a given node, 76% of the code flips in which every serialization is answered (398 of 525) contain exactly the other reading's answer, reachable nodes instead of neighbours or the reverse. On the Erdős `neighbor` task, all 84 such flips of Qwen3-8B contain a strict subset of the true neighbours, because the program records each edge in one direction only, as if the graph were directed.

Finally, our first Graph-as-Code run on Erdős did not state that the graph is undirected. Where the task's preamble names "directed" only to define a conditional that does not apply (`neighbor`), *all four models* built a directed graph on every undirected instance they could be observed on, and every arm scored 28/100. Stating the type gives 100/100.

5.4 Keeping the graph out of the prompt

The most direct way to avoid the sensitivity is not to show the model the serialization at all. In Graph-as-Code (M3) the model writes its program from the task and the question alone, and the graph reaches the program as data. This costs no accuracy: with plain Python, M3 is the most accurate mode on both datasets (Table 3). It answers 92.9% of GraphQA and 97.5% of Erdős questions correctly, against 88.3% and 96.9% for code from text, and it also has the highest robust accuracy.

Without the graph text, the prompt itself must say whether the graph is directed (Section 5.3), so Table 3 reports M3 on Erdős from a rerun that states the type in every undirected prompt (GraphQA's prompts already do). Table D1 in Appendix D gives the first run.

	M1	M2		M3	
		Py	NX	Py	NX
Accuracy, all serializations					
GraphQA	84.4	88.3	88.8	**92.9**	87.0
Erdős	87.0	96.9	91.1	**97.5**	89.1
Robust accuracy, relabeling and order					
GraphQA	76.7	87.2	85.4	**92.2**	86.8
Erdős	78.6	95.9	90.2	**97.3**	89.0

Table 3: Accuracy (% of answers) and robust accuracy (% of tuples correct on every serialization), models pooled. M3 on Erdős states the graph type. Bold: row maximum.

Two limits remain: with NetworkX, M3 stays about 2 points below code from text in accuracy, and its errors are systematic: when the question names no node, one program answers every instance of a task (Appendix C), so a wrong program is wrong on every graph and every serialization.

6 Conclusion and Limitations

Executed code keeps its answer far more often than direct answering under relabeling and edge order, and its advantage grows with the size of the graph, but it is not invariant, and not under every factor. Listing each edge in both directions brings plain-Python code close to direct answering, and writing the graph as NetworkX code makes two models' NetworkX programs crash, because they act as if the prompt's code had already run. Graphs are transcribed almost perfectly, and the flips start in the program the model writes, which often answers a different reading of the question.

Keeping the graph out of the prompt nearly removes the sensitivity at no cost in accuracy if the prompt states the graph type, though its errors become systematic.

Our study has several limits. We test four models through one gateway and cannot control how deterministic the providers serving them are: even at temperature 0, an identical prompt sometimes gets a different answer, so part of every flip rate is noise that repeated prompts can only estimate. Every experiment but the repeat baseline uses one generation per prompt. The graphs are small (5-20 and 4-34 nodes), and we use 12 of Erdős's 50 tasks. The prompts are zero-shot, with reasoning turned off and NetworkX permitted, not required,

so reasoning models or other prompts may behave differently. Finally, each serialization changes one factor at a time, so we do not test factors in combination.

References

Qiaolong Cai, Zhaowei Wang, Shizhe Diao, James Kwok, and Yangqiu Song. 2024. CodeGraph: Enhancing graph reasoning of LLMs with code. *Preprint*, arXiv:2408.13863.

Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2024. Talk like a graph: Encoding graphs for large language models. In *The Twelfth International Conference on Learning Representations*.

Ben Finkelshtein, Silviu Cucerzan, Sujay Kumar Jauhar, and Ryen White. 2026. Actions speak louder than prompts: A large-scale study of LLMs for graph inference. In *The Fourteenth International Conference on Learning Representations*.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10764–10799.

Yuyao Ge, Shenghua Liu, Baolong Bi, Yiwei Wang, Lingrui Mei, Wenjie Feng, Lizhe Chen, and Xueqi Cheng. 2025. Can graph descriptive order affect solving graph problems with LLMs? In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*.

Xiaojun Guo, Ang Li, Yifei Wang, Stefanie Jegelka, and Yisen Wang. 2025. G1: Teaching LLMs to reason on graphs with reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 38.

Daniel Herbst, Lea Karbevska, Divyanshu Kumar, Akanksha Ahuja, Fatemeh Gholamzadeh Nasrabadi, and Fabrizio Frasca. 2026. Lost in serialization: Invariance and generalization of LLM graph reasoners. In *ICML Workshop on Graph Foundation Models*.

Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. 2023. Can language models solve graph problems in natural language? In *Advances in Neural Information Processing Systems*, volume 36.

Appendix

A Prompt Examples

M1 (direct answering)

```
In an undirected graph, (i,j) means that node i and node j are connected with an undirected edge. G describes a graph among nodes
0, 1, 2, 3, and 4.
The edges in G are: (1, 2) (1, 4) (2, 3).

Question: What is the degree of node 3?

Your answer should be an integer.

Solve the above problem efficiently and clearly. The last line of your response should be of the following format: 'Therefore, the
 final answer is: $\boxed{ANSWER}$.'
```

M2 (code from text), plain-Python arm

```
For this task, please calculate the degree of a given node in the undirected graph G.
Write a piece of Python code to return the answer in a variable 'ans'. Please enclose the code with # CODE START and # CODE END.

Fill in the template below and keep its structure. 'nodes' must list every node of G and 'edges' must list every edge of G as (u,
v) pairs. Then write the code that computes the answer from 'nodes' and 'edges'. Compute the result - do not write it directly.

# CODE START
nodes = <the nodes of G>
edges = <the edges of G, as (u, v) pairs>

<your code here>
ans = <the computed result, a Python int>
# CODE END

Use only plain Python and its standard library. Do not import networkx or any other third-party library.

Q: What is the degree of node 3?
In an undirected graph, (i,j) means that node i and node j are connected with an undirected edge. G describes a graph among nodes
0, 1, 2, 3, and 4.
The edges in G are: (1, 2) (1, 4) (2, 3).
A:
```

M3 (Graph-as-Code), plain-Python arm: the parts that differ from M2

```
[... first two lines as in M2 ...]

Fill in the template below and keep its structure. The variables 'nodes' (the nodes of G) and 'edges' (the edges of G, as (u, v)
pairs) are already defined; use them directly and do not redefine them. Compute the result - do not write it directly.

# CODE START
# 'nodes' and 'edges' are already defined.

<your code here>
ans = <the computed result, a Python int>
# CODE END

[... library line as in M2 ...]

Q: What is the degree of node 3?
A:
```

Defined by the harness before the M3 program runs:

```
nodes = [0, 1, 2, 3, 4]
edges = [(1, 2), (1, 4), (2, 3)]
```

Figure A1: The prompts for one GraphQA instance, a 5-node graph with the question “What is the degree of node 3?” (answer 1), in the plain-Python arm. M1 uses the system prompt “You are a helpful assistant.”. M2 and M3 use one describing an expert in graph networks and Python. In the NetworkX arm, the library line reads “You may use the networkx library.”

B Additional Results

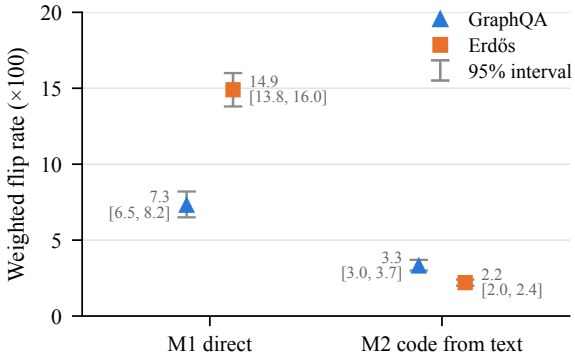


Figure B1: Weighted flip rate (mean D over all tuples) of M1 and M2 under relabeling and edge order. M2 is averaged over plain Python and NetworkX. Whiskers are 95% confidence intervals over the choice of graphs, computed as in Figure 1.

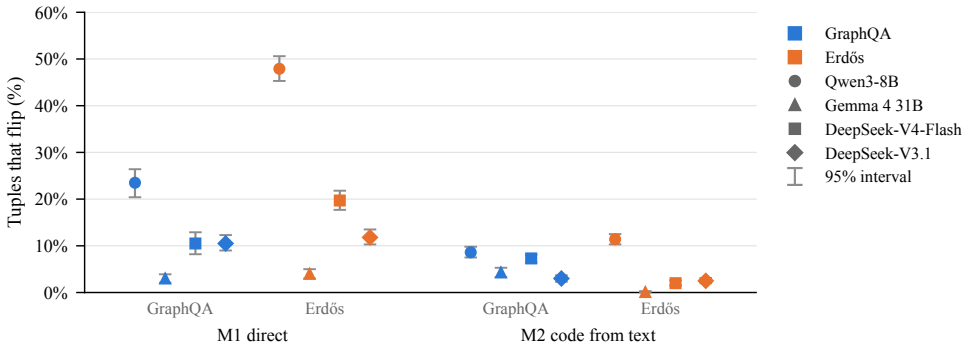


Figure B2: Flip rate of M1 and M2 for each model under relabeling and edge order, as in Figure 1 but without pooling the models. Color marks the dataset and shape the model. M2 is averaged over plain Python and NetworkX. Whiskers are 95% confidence intervals over the choice of graphs, computed as in Figure 1.

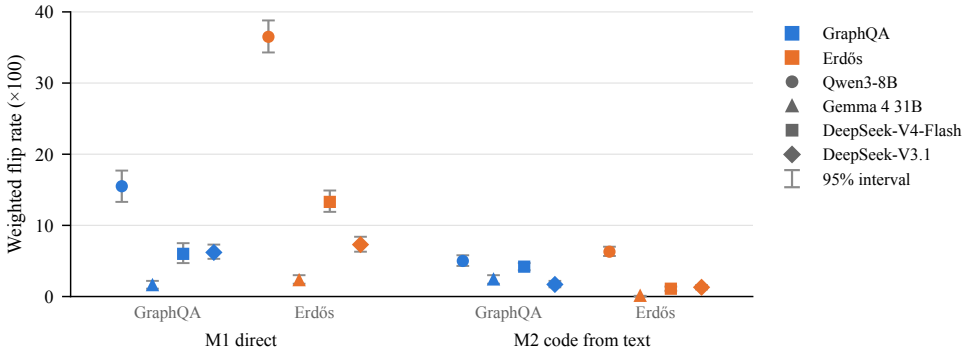


Figure B3: Weighted flip rate (mean D over all tuples) for each model under relabeling and edge order, as in Figure B1. Whiskers as in Figure B2.

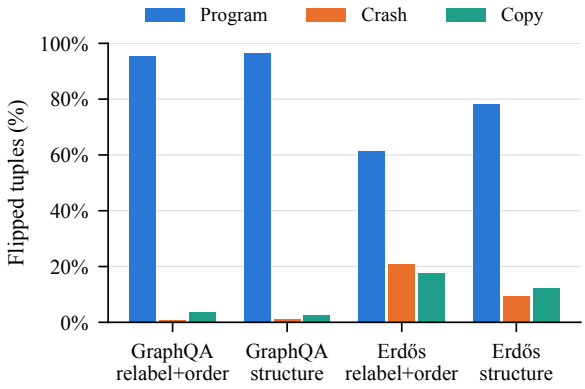


Figure B4: Where plain-Python flips start, four models pooled.

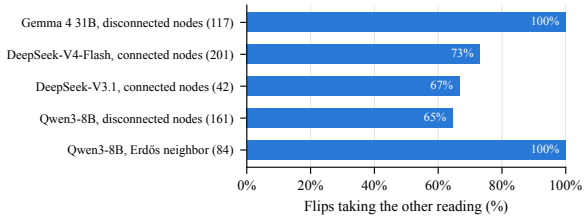


Figure B5: Share of code flips that contain the answer to the other reading of the question, under relabeling and order, among flips in which every serialization is answered.

	GraphQA			Erdős		
Bucket	Dir.	Py	NX	Dir.	Py	NX
Qwen3-8B						
(1) Same, correct	69.8	93.8	84.5	48.5	84.6	60.7
(2) Same, wrong	6.6	0.4	3.7	1.0	1.8	0.9
(3) Different, all correct	0.0	0.0	0.0	2.6	4.8	4.5
(4) Flip, all answered	22.2	5.6	9.1	42.4	7.8	1.2
(5) Flip with a crash	1.2	0.1	2.3	5.5	1.0	12.8
(6) No answer on all	0.1	0.0	0.4	0.0	0.0	20.0
Gemma 4 31B						
(1) Same, correct	87.5	95.3	94.6	92.2	95.5	95.6
(2) Same, wrong	9.5	1.0	0.5	0.1	0.0	0.0
(3) Different, all correct	0.0	0.0	0.0	3.7	4.4	4.2
(4) Flip, all answered	3.0	3.7	4.8	3.4	0.1	0.0
(5) Flip with a crash	0.0	0.0	0.1	0.6	0.1	0.1
(6) No answer on all	0.0	0.0	0.0	0.0	0.0	0.0
DeepSeek-V4-Flash						
(1) Same, correct	75.7	74.9	78.6	75.3	91.5	94.8
(2) Same, wrong	13.8	18.0	13.9	0.7	0.6	0.4
(3) Different, all correct	0.0	0.0	0.0	4.3	4.4	4.2
(4) Flip, all answered	10.5	7.1	7.5	17.9	1.3	0.6
(5) Flip with a crash	0.0	0.0	0.0	1.8	2.2	0.0
(6) No answer on all	0.0	0.0	0.0	0.0	0.0	0.0
DeepSeek-V3.1						
(1) Same, correct	73.7	84.6	83.9	83.5	93.8	92.3
(2) Same, wrong	15.8	12.8	12.7	0.2	0.0	0.0
(3) Different, all correct	0.0	0.0	0.0	4.4	4.6	4.4
(4) Flip, all answered	10.4	2.6	0.9	6.9	1.5	2.0
(5) Flip with a crash	0.1	0.0	2.4	4.9	0.1	1.3
(6) No answer on all	0.0	0.0	0.1	0.0	0.0	0.0

Table B1: Tuples by bucket for each model under relabeling and edge order, as in Table 1 (% of tuples). Dir. = M1. Py and NX = M2 in plain Python and with NetworkX.

	GraphQA			Erdős		
Factor	Dir.	Py	NX	Dir.	Py	NX
Qwen3-8B						
Relabeling	**24.3**	7.3	**12.7**	**49.2**	8.9	**14.4**
Order	**22.6**	4.2	**10.0**	**46.7**	8.8	**13.6**
Structure	**32.0**	**40.3**†	**18.7**	**49.5**	**35.1**	**28.5**
Syntax	**27.4**	4.9	**39.3**†	**49.1**	9.9	**34.2**
Gemma 4 31B						
Relabeling	3.3	4.1	5.6†	4.3	0.2	0.2
Order	2.8	3.2	4.2†	3.6	0.2	0.0
Structure	9.3	2.7	3.1	3.3	0.9	1.4
Syntax	6.9	3.6	4.3	2.6	0.4	0.8
DeepSeek-V4-Flash						
Relabeling	**11.9**	7.9	7.3	**21.9**	3.4	0.6
Order	9.1	6.2	7.7	**17.4**	3.6	0.6
Structure	**16.9**	**17.0**	7.3	**15.8**	**11.4**	7.8
Syntax	**13.3**	7.6	6.7	**15.8**	4.3	**29.7**†
DeepSeek-V3.1						
Relabeling	**11.0**	3.0	3.3	**13.0**	1.8	3.4
Order	**10.0**	2.2	3.3	**10.7**	1.5	3.2
Structure	**18.7**	**14.0**	2.9	9.4	7.4	3.1
Syntax	**12.0**	3.3	4.1	8.2	0.7	1.6

Table B2: Flip rate by factor for each model, as in Table 2 (% of tuples). Bold: 10% or more. †: the model's code flips at least 1 point more often than its own direct answers.

		Direct		Native		NetworkX	
Dataset	Model	I	R	I	R	I	R
GraphQA	Qwen3-8B	76	70	94	94	88	85
GraphQA	Gemma 4 31B	97	87	96	95	95	95
GraphQA	DeepSeek-V4-F	90	76	93	75	92	79
GraphQA	DeepSeek-V3.1	89	74	97	85	97	84
Erdős	Qwen3-8B	50	51	86	89	62	65
Erdős	Gemma 4 31B	92	96	95	100	96	100
Erdős	DeepSeek-V4-F	76	80	92	96	95	99
Erdős	DeepSeek-V3.1	84	88	94	98	92	97

Table B3: Proposed for removal: this table uses the earlier consistency measure I (literally identical answers), which the paper no longer defines. Table B1 gives the same comparison with the current buckets.

C Setup and Metric Details

Serializations. Relabeling sorts the renamed edge list again and compares it against the original labeling, sorted the same way, so both follow the same ordering rule. The two sorted orderings break ties in random order. On Erdős, whose original edge list is not sorted, full sorting replaces the second-endpoint ordering. The adjacency list has one line per node followed by its neighbours. For GraphQA this is the paper's "incident" encoding. The NetworkX-code syntax is code that builds the graph.

Failure labels. The copy step of the localization (Section 4.2) is approximate: declarations are read after execution, and the comparison ignores edge weights and multiplicity.

Inference. Qwen3-8B, Gemma and DeepSeek-V4-Flash have a switch for reasoning, and DeepSeek-V3.1 answers without reasoning by default. Besides keeping direct answers direct, turning reasoning off keeps a model from spending the whole token budget before it answers: Qwen3-8B with reasoning on often returned no answer at all.

Repeated prompts. Two serializations sometimes produce exactly the same prompt. This is common in M3, whose prompt holds the question but not the graph: when the question names no node, every serialization of an instance, and every instance of the task, gives the same M3 prompt. We send each distinct prompt once and reuse its reply, so in M3 one program can answer many graphs. Only 56-57% of all prompts are separate model calls.

Correctness. Density and diameter on Erdős are real numbers, so there we accept an answer within 0.01 of the true value (1% for values above 1), so that a density written as 0.27 instead of 0.2727 counts.

Spread. S uses only the answers that exist and divides by 1 when $y = 0$.

Statistics. McNemar's exact test (flips) and the Wilcoxon signed-rank test (D) are run for each model, dataset and library and Holm-corrected over the 16 cases. The logistic regression, one per dataset, models flipping on mode, factor, model and task, with errors clustered by graph. Bootstrap intervals resample the graphs of each dataset with replacement 2,000 times. The repeat floor reports how often the answer to an identical prompt changes, under the same flip rule: Qwen3-8B's code exceeds it in all 8 cells, by 4.0-11.0 points, while the other three models' code does so in only 3 of 24 (95% Wald interval on the excess above 0), so their small residual is close to noise.

Factor	Answers in a tuple	GraphQA	Erdős
Relabeling	reference labeling + 3 random relabelings	4 (699 tuples), 3 (1)	4 (1,199), 3 (1)
Order	dataset's own order + 3 reorderings	4 (587), 3 (63), 2 (39)	4 (1,133), 3 (58), 2 (9)
Structure	reference + adjacency list + each edge in both directions	3 (700)	3 (1,123), 2 (77)
Syntax	reference + plain text, JSON, NetworkX code	4 (700)	3, JSON and NetworkX code only (1,200)

Table C1: Answers per tuple, with the number of tuples of each size per model and mode. A serialization identical to its reference is dropped, which shortens some tuples: 11 GraphQA instances have no order tuple, one random relabeling per dataset came out as the identity, and the 77 directed Erdős graphs have no both-directions serialization.

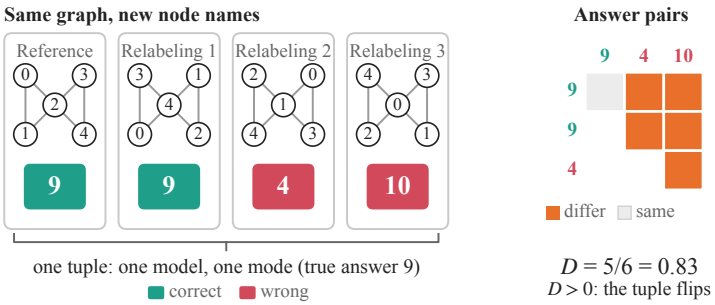


Figure C1: One tuple: Qwen3-8B answering Erdős triangles-0000 directly under four labelings of one graph (drawn schematically). Green answers are correct, red wrong. Five of the six answer pairs differ, so $D = 0.83$.

D M3 Without the Stated Graph Type

Table D1 repeats Table 3 with the first M3 run on Erdős, whose prompts did not state that the graph is undirected (Section 5.4). Plain-Python M3 is then 5.7 points below code from text on Erdős, almost all of it on `neighbor`. The M3 control in Section 5.3 uses this first run.

The omission is also partly invisible: in the first run, 12 of the 84 undirected `neighbor` answers were still correct. The rerun adds "The graph is undirected: (u, v) and (v, u) denote the same edge." to every undirected M3 prompt. Stating the type raises `neighbor` from 28 to 100 of 100 for every model and library and also fixes two single-model misreadings, DeepSeek-V3.1's `has_cycle` and Gemma's `edge_existence`. Because every M3 program was generated afresh, a few tasks answered by a single program also moved by chance in both directions (for example `bridges`). In the first run, where the preamble asserts that the graph is undirected, no program built a directed graph, and where it says nothing only one did. Where it describes both cases, Gemma built a directed graph for 69% of recovered graphs, nearly three quarters still correct.

	M1	M2		M3	
		Py	NX	Py	NX
Accuracy, all serializations					
GraphQA	84.4	88.3	88.8	**92.9**	87.0
Erdős	87.0	**96.9**	91.1	91.2	84.2
Robust accuracy, relabeling and order					
GraphQA	76.7	87.2	85.4	**92.2**	86.8
Erdős	78.6	**95.9**	90.2	91.0	84.1

Table D1: Table 3 with the first M3 run on Erdős, before the graph type was stated in its prompt. M1, M2 and all of GraphQA are unchanged. Bold: highest in the row.